File: Makefile

```
BINARIES = main-two-cvs-while main-two-cvs-if main-one-cv-while
HEADERS = mythreads.h main-header.h main-common.c pc-header.h

all: $(BINARIES)

clean:
■rm -f $(BINARIES)

main-one-cv-while: main-one-cv-while.c $(HEADERS)
■gcc -o main-one-cv-while main-one-cv-while.c -Wall -Wno-pointer-to-int-cast -pthread

main-two-cvs-if: main-two-cvs-if.c $(HEADERS)
■gcc -o main-two-cvs-if main-two-cvs-if.c -Wall -Wno-pointer-to-int-cast -pthread

main-two-cvs-while: main-two-cvs-while.c $(HEADERS)
■gcc -o main-two-cvs-while main-two-cvs-while.c -Wall -Wno-pointer-to-int-cast -pthread
```

File: main-common.c

```
// Common usage() prints out stuff for command-line help
void usage() {
    fprintf(stderr, &amp;quot;usage: \n&amp;quot;);
    fprintf(stderr, &amp;quot;  -l &amp;lt;number of items each producer produces&amp;gt;\n&amp;quot;);
    fprintf(stderr, &amp;quot;  -m &amp;lt;size of the shared producer/consumer buffer&amp;gt;\n&amp;quot;);
    fprintf(stderr, &amp;quot;  -p &amp;lt;number of producers&amp;gt;\n&amp;quot;);
    fprintf(stderr, &amp;quot;  -c &amp;lt;number of consumers&amp;gt;\n&amp;quot;);
    fprintf(stderr, &amp;quot;  -P &amp;lt;sleep string: how each producer should sleep at various points in execution&amp;g
    fprintf(stderr, &amp;quot;  -C &amp;lt;sleep string: how each consumer should sleep at various points in execution&amp;g
    fprintf(stderr, &amp;quot;  -v [ verbose flag: trace what is happening and print it ]\n&amp;quot;);
    fprintf(stderr, &amp;quot;  -t [ timing flag: time entire execution and print total time ]\n&amp;quot;);
    exit(1);
}

// Common main() for all four programs
// - Does arg parsing
// - Starts producers and consumers
// - Once producers are finished, puts END_OF_STREAM
//   marker into shared queue to signal end to consumers
// - Then waits for consumers and prints some final info
int main(int argc, char *argv[]) {
    loops = 1;
    max = 1;
    consumers = 1;
    producers = 1;

    char *producer_pause_string = NULL;
    char *consumer_pause_string = NULL;

    opterr = 0;
    int c;
    while ((c = getopt (argc, argv, &amp;quot;l:m:p:c:P:C:vt&amp;quot;)) != -1) {
■switch (c) {
■case &amp;#x27;l&amp;#x27;:
■    loops = atoi(optarg);
■    break;
■case &amp;#x27;m&amp;#x27;:
■    max = atoi(optarg);
■    break;
■case &amp;#x27;p&amp;#x27;:
■    producers = atoi(optarg);
■    break;
■case &amp;#x27;c&amp;#x27;:
■    consumers = atoi(optarg);
■    break;
■case &amp;#x27;P&amp;#x27;:
■    producer_pause_string = optarg;
■    break;
■case &amp;#x27;C&amp;#x27;:
■    consumer_pause_string = optarg;
■    break;
■case &amp;#x27;v&amp;#x27;:
■    do_trace = 1;
■    break;
■case &amp;#x27;t&amp;#x27;:
■    do_timing = 1;
■    break;
■default:
■    usage();
■}
    }

    assert(loops &amp;gt; 0);
    assert(max &amp;gt; 0);
    assert(producers &amp;lt;= MAX_THREADS);
    assert(consumers &amp;lt;= MAX_THREADS);

    if (producer_pause_string != NULL)
```

```
■parse_pause_string(producer_pause_string, &amp;quot;producers&amp;quot;, producers, producer_pause_times);
    if (consumer_pause_string != NULL)
■parse_pause_string(consumer_pause_string, &amp;quot;consumers&amp;quot;, consumers, consumer_pause_times);

    // make space for shared buffer, and init it ...
    buffer = (int *) Malloc(max * sizeof(int));
    int i;
    for (i = 0; i &amp;lt; max; i++) {
■buffer[i] = EMPTY;
    }

    do_print_headers();

    double t1 = Time_GetSeconds();

    // start up all threads; order doesn&amp;#x27;t matter here
    pthread_t pid[MAX_THREADS], cid[MAX_THREADS];
    int thread_id = 0;
    for (i = 0; i &amp;lt; producers; i++) {
■Pthread_create(&amp;amp;pid[i], NULL, producer, (void *) (long long) thread_id);
■thread_id++;
    }
    for (i = 0; i &amp;lt; consumers; i++) {
■Pthread_create(&amp;amp;cid[i], NULL, consumer, (void *) (long long) thread_id);
■thread_id++;
    }

    // now wait for all PRODUCERS to finish
    for (i = 0; i &amp;lt; producers; i++) {
■Pthread_join(pid[i], NULL);
    }

    // end case: when producers are all done
    // - put &amp;quot;consumers&amp;quot; number of END_OF_STREAM&amp;#x27;s in queue
    // - when consumer sees -1, it exits
    for (i = 0; i &amp;lt; consumers; i++) {
■Mutex_lock(&amp;amp;m);
■while (num_full == max)
■    Cond_wait(empty_cv, &amp;amp;m);
■do_fill(END_OF_STREAM);
■do_eos();
■Cond_signal(fill_cv);
■Mutex_unlock(&amp;amp;m);
    }

    // now OK to wait for all consumers
    int counts[consumers];
    for (i = 0; i &amp;lt; consumers; i++) {
■Pthread_join(cid[i], (void *) &amp;amp;counts[i]);
    }

    double t2 = Time_GetSeconds();

    if (do_trace) {
■printf(&amp;quot;\nConsumer consumption:\n&amp;quot;);
■for (i = 0; i &amp;lt; consumers; i++) {
■    printf(&amp;quot;  C%d -&amp;gt; %d\n&amp;quot;, i, counts[i]);
■}
■printf(&amp;quot;\n&amp;quot;);
    }

    if (do_timing) {
■printf(&amp;quot;Total time: %.2f seconds\n&amp;quot;, t2-t1);
    }

    return 0;
}
```

File: main-header.h

```
#ifndef __main_header_h__
#define __main_header_h__

// all this is for the homework side of things

int do_trace = 0;
int do_timing = 0;

#define p0 do_pause(id, 1, 0, &amp;quot;p0&amp;quot;);
#define p1 do_pause(id, 1, 1, &amp;quot;p1&amp;quot;);
#define p2 do_pause(id, 1, 2, &amp;quot;p2&amp;quot;);
#define p3 do_pause(id, 1, 3, &amp;quot;p3&amp;quot;);
#define p4 do_pause(id, 1, 4, &amp;quot;p4&amp;quot;);
#define p5 do_pause(id, 1, 5, &amp;quot;p5&amp;quot;);
#define p6 do_pause(id, 1, 6, &amp;quot;p6&amp;quot;);

#define c0 do_pause(id, 0, 0, &amp;quot;c0&amp;quot;);
#define c1 do_pause(id, 0, 1, &amp;quot;c1&amp;quot;);
#define c2 do_pause(id, 0, 2, &amp;quot;c2&amp;quot;);
#define c3 do_pause(id, 0, 3, &amp;quot;c3&amp;quot;);
#define c4 do_pause(id, 0, 4, &amp;quot;c4&amp;quot;);
#define c5 do_pause(id, 0, 5, &amp;quot;c5&amp;quot;);
#define c6 do_pause(id, 0, 6, &amp;quot;c6&amp;quot;);

int producer_pause_times[MAX_THREADS][7];
int consumer_pause_times[MAX_THREADS][7];

// needed to avoid interleaving of print out from threads
pthread_mutex_t print_lock = PTHREAD_MUTEX_INITIALIZER;

void do_print_headers() {
    if (do_trace == 0)
■return;
    int i;
    printf(&amp;quot;%3s &amp;quot;, &amp;quot;NF&amp;quot;);
    for (i = 0; i &amp;lt; max; i++) {
■printf(&amp;quot; %3s &amp;quot;, &amp;quot;   &amp;quot;);
    }
    printf(&amp;quot;   &amp;quot;);

    for (i = 0; i &amp;lt; producers; i++)
■printf(&amp;quot;P%d &amp;quot;, i);
    for (i = 0; i &amp;lt; consumers; i++)
■printf(&amp;quot;C%d &amp;quot;, i);
    printf(&amp;quot;\n&amp;quot;);
}

void do_print_pointers(int index) {
    if (use_ptr == index &amp;amp;&amp;amp; fill_ptr == index) {
■printf(&amp;quot;*&amp;quot;);
    } else if (use_ptr == index) {
■printf(&amp;quot;u&amp;quot;);
    } else if (fill_ptr == index) {
■printf(&amp;quot;f&amp;quot;);
    } else {
■printf(&amp;quot; &amp;quot;);
    }
}

void do_print_buffer() {
    int i;
    printf(&amp;quot;%3d [&amp;quot;, num_full);
    for (i = 0; i &amp;lt; max; i++) {
■do_print_pointers(i);
■if (buffer[i] == EMPTY) {
■    printf(&amp;quot;%3s &amp;quot;, &amp;quot;---&amp;quot;);
■} else if (buffer[i] == END_OF_STREAM) {
■    printf(&amp;quot;%3s &amp;quot;, &amp;quot;EOS&amp;quot;);
```

```
■} else {
■    printf(&quot;%3d &quot;, buffer[i]);
■}
    }
    printf(&quot;] &quot;);
}

void do_eos() {
    if (do_trace) {
■Mutex_lock(&amp;print_lock);
■do_print_buffer();
■//printf(&quot;%3d [added end-of-stream marker]\n&quot;, num_full);
■printf(&quot;[main: added end-of-stream marker]\n&quot;);
■Mutex_unlock(&amp;print_lock);
    }
}

void do_pause(int thread_id, int is_producer, int pause_slot, char *str) {
    int i;
    if (do_trace) {
■Mutex_lock(&amp;print_lock);
■do_print_buffer();

■// skip over other thread&#x27;s spots
■for (i = 0; i &lt; thread_id; i++) {
■    printf(&quot;   &quot;);
■}
■printf(&quot;%s\n&quot;, str);
■Mutex_unlock(&amp;print_lock);
    }

    int local_id = thread_id;
    int pause_time;
    if (is_producer) {
■pause_time = producer_pause_times[local_id][pause_slot];
    } else {
■local_id = thread_id - producers;
■pause_time = consumer_pause_times[local_id][pause_slot];
    }
    // printf(&quot; PAUSE %d\n&quot;, pause_time);
    sleep(pause_time);
}

void ensure(int expression, char *msg) {
    if (expression == 0) {
■fprintf(stderr, &quot;%s\n&quot;, msg);
■exit(1);
    }
}

void parse_pause_string(char *str, char *name, int expected_pieces,
■■■int pause_array[MAX_THREADS][7]) {

    // string looks like this (or should):
    //   1,2,0:2,3,4,5
    //   n-1 colons if there are n producers/consumers
    //   comma-separated for sleep amounts per producer or consumer
    int index = 0;

    char *copy_entire = strdup(str);
    char *outer_marker;
    int colon_count = 0;
    char *p = strtok_r(copy_entire, &quot;:&quot;, &amp;outer_marker);
    while (p) {
■// init array: default sleep is 0
■int i;
■for (i = 0; i &lt; 7; i++)
■    pause_array[index][i] = 0;

■// for each piece, comma separated
```

```
■char *inner_marker;
■char *copy_piece = strdup(p);
■char *c = strtok_r(copy_piece, &amp;quot;,&amp;quot;, &amp;amp;inner_marker);
■int comma_count = 0;

■int inner_index = 0;
■while (c) {
■    int pause_amount = atoi(c);
■    ensure(inner_index &amp;lt; 7, &amp;quot;you specified a sleep string incorrectly... (too many comma-separated args
■    // printf(&amp;quot;setting %s pause %d to %d\n&amp;quot;, name, inner_index, pause_amount);
■    pause_array[index][inner_index] = pause_amount;
■    inner_index++;

■    c = strtok_r(NULL, &amp;quot;,&amp;quot;, &amp;amp;inner_marker);
■    comma_count++;
■}
■free(copy_piece);
■index++;

■// continue with colon separated list
■p = strtok_r(NULL, &amp;quot;:&amp;quot;, &amp;amp;outer_marker);
■colon_count++;
    }

    free(copy_entire);
    if (expected_pieces != colon_count) {
■fprintf(stderr, &amp;quot;Error: expected %d %s in sleep specification, got %d\n&amp;quot;, expected_pieces, name, colo
■exit(1);
    }
}


#endif // __main_header_h__
```

File: main-one-cv-while.c

```
#include &lt;ctype.h&gt;
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;unistd.h&gt;
#include &lt;assert.h&gt;
#include &lt;pthread.h&gt;
#include &lt;sys/time.h&gt;
#include &lt;string.h&gt;

#include &quot;mythreads.h&quot;

#include &quot;pc-header.h&quot;

pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

#include &quot;main-header.h&quot;

void do_fill(int value) {
    // ensure empty before usage
    ensure(buffer[fill_ptr] == EMPTY, &quot;error: tried to fill a non-empty buffer&quot;);
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % max;
    num_full++;
}

int do_get() {
    int tmp = buffer[use_ptr];
    ensure(tmp != EMPTY, &quot;error: tried to get an empty buffer&quot;);
    buffer[use_ptr] = EMPTY;
    use_ptr = (use_ptr + 1) % max;
    num_full--;
    return tmp;
}

void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i &lt; loops; i++) {   p0;
■Mutex_lock(&amp;m);             p1;
■while (num_full == max) {   p2;
■    Cond_wait(&amp;cv, &amp;m);     p3;
■}
■do_fill(base + i);          p4;
■Cond_signal(&amp;cv);           p5;
■Mutex_unlock(&amp;m);           p6;
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
■Mutex_lock(&amp;m);            c1;
■while (num_full == 0) {    c2;
■    Cond_wait(&amp;cv, &amp;m);    c3;
        }
■tmp = do_get();            c4;
■Cond_signal(&amp;cv);          c5;
■Mutex_unlock(&amp;m);          c6;
■consumed_count++;
    }

    // return consumer_count-1 because END_OF_STREAM does not count
```

```
    return (void *) (long long) (consumed_count - 1);
}

// must set these appropriately to use &amp;quot;main-common.c&amp;quot;
pthread_cond_t *fill_cv = &amp;amp;cv;
pthread_cond_t *empty_cv = &amp;amp;cv;

// all codes use this common base to start producers/consumers
// and all the other related stuff
#include &amp;quot;main-common.c&amp;quot;
```

File: main-two-cvs-if.c

```
#include &lt;ctype.h&gt;
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;unistd.h&gt;
#include &lt;assert.h&gt;
#include &lt;pthread.h&gt;
#include &lt;sys/time.h&gt;
#include &lt;string.h&gt;

#include &quot;mythreads.h&quot;
#include &quot;pc-header.h&quot;

pthread_cond_t empty  = PTHREAD_COND_INITIALIZER;
pthread_cond_t fill   = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m     = PTHREAD_MUTEX_INITIALIZER;

#include &quot;main-header.h&quot;

void do_fill(int value) {
    // ensure empty before usage
    ensure(buffer[fill_ptr] == EMPTY, &quot;error: tried to fill a non-empty buffer&quot;);
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % max;
    num_full++;
}

int do_get() {
    int tmp = buffer[use_ptr];
    ensure(tmp != EMPTY, &quot;error: tried to get an empty buffer&quot;);
    buffer[use_ptr] = EMPTY;
    use_ptr = (use_ptr + 1) % max;
    num_full--;
    return tmp;
}

void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i &lt; loops; i++) {   p0;
■Mutex_lock(&amp;m);             p1;
■if (num_full == max) {      p2;
■    Cond_wait(&amp;empty, &amp;m);  p3;
■}
■do_fill(base + i);          p4;
■Cond_signal(&amp;fill);         p5;
■Mutex_unlock(&amp;m);           p6;
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
■Mutex_lock(&amp;m);            c1;
■if (num_full == 0) {       c2;
■    Cond_wait(&amp;fill, &amp;m);  c3;
        }
■tmp = do_get();            c4;
■Cond_signal(&amp;empty);       c5;
■Mutex_unlock(&amp;m);          c6;
■consumed_count++;
    }

    // return consumer_count-1 because END_OF_STREAM does not count
```

```
    return (void *) (long long) (consumed_count - 1);
}

// must set these appropriately to use &amp;quot;main-common.c&amp;quot;
pthread_cond_t *fill_cv = &amp;amp;fill;
pthread_cond_t *empty_cv = &amp;amp;empty;

// all codes use this common base to start producers/consumers
// and all the other related stuff
#include &amp;quot;main-common.c&amp;quot;
```

File: main-two-cvs-while.c

```
#include &lt;ctype.h&gt;
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;unistd.h&gt;
#include &lt;assert.h&gt;
#include &lt;pthread.h&gt;
#include &lt;sys/time.h&gt;
#include &lt;string.h&gt;

#include &quot;mythreads.h&quot;

#include &quot;pc-header.h&quot;

// used in producer/consumer signaling protocol
pthread_cond_t empty  = PTHREAD_COND_INITIALIZER;
pthread_cond_t fill   = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m     = PTHREAD_MUTEX_INITIALIZER;

#include &quot;main-header.h&quot;

void do_fill(int value) {
    // ensure empty before usage
    ensure(buffer[fill_ptr] == EMPTY, &quot;error: tried to fill a non-empty buffer&quot;);
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % max;
    num_full++;
}

int do_get() {
    int tmp = buffer[use_ptr];
    ensure(tmp != EMPTY, &quot;error: tried to get an empty buffer&quot;);
    buffer[use_ptr] = EMPTY;
    use_ptr = (use_ptr + 1) % max;
    num_full--;
    return tmp;
}

void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i &lt; loops; i++) {   p0;
■Mutex_lock(&amp;m);             p1;
■while (num_full == max) {   p2;
■    Cond_wait(&amp;empty, &amp;m);  p3;
■}
■do_fill(base + i);          p4;
■Cond_signal(&amp;fill);         p5;
■Mutex_unlock(&amp;m);           p6;
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
■Mutex_lock(&amp;m);            c1;
■while (num_full == 0) {    c2;
■    Cond_wait(&amp;fill, &amp;m);  c3;
        }
■tmp = do_get();            c4;
■Cond_signal(&amp;empty);       c5;
■Mutex_unlock(&amp;m);          c6;
■consumed_count++;
    }
```

```
    // return consumer_count-1 because END_OF_STREAM does not count
    return (void *) (long long) (consumed_count - 1);
}

// must set these appropriately to use &amp;quot;main-common.c&amp;quot;
pthread_cond_t *fill_cv = &amp;amp;fill;
pthread_cond_t *empty_cv = &amp;amp;empty;

// all codes use this common base to start producers/consumers
// and all the other related stuff
#include &amp;quot;main-common.c&amp;quot;
```

File: mythreads.h

```
#ifndef __MYTHREADS_h__
#define __MYTHREADS_h__

#include &lt;pthread.h&gt;
#include &lt;assert.h&gt;
#include &lt;sys/time.h&gt;
#include &lt;stdlib.h&gt;

void *Malloc(size_t size) {
    void *p = malloc(size);
    assert(p != NULL);
    return p;
}

double Time_GetSeconds() {
    struct timeval t;
    int rc = gettimeofday(&amp;t, NULL);
    assert(rc == 0);
    return (double) ((double)t.tv_sec + (double)t.tv_usec / 1e6);
}

void work(int seconds) {
    double t0 = Time_GetSeconds();
    while ((Time_GetSeconds() - t0) &lt; (double)seconds)
■;
}

void Mutex_init(pthread_mutex_t *m) {
    assert(pthread_mutex_init(m, NULL) == 0);
}

void Mutex_lock(pthread_mutex_t *m) {
    int rc = pthread_mutex_lock(m);
    assert(rc == 0);
}

void Mutex_unlock(pthread_mutex_t *m) {
    int rc = pthread_mutex_unlock(m);
    assert(rc == 0);
}

void Cond_init(pthread_cond_t *c) {
    assert(pthread_cond_init(c, NULL) == 0);
}

void Cond_wait(pthread_cond_t *c, pthread_mutex_t *m) {
    int rc = pthread_cond_wait(c, m);
    assert(rc == 0);
}

void Cond_signal(pthread_cond_t *c) {
    int rc = pthread_cond_signal(c);
    assert(rc == 0);
}


void Pthread_create(pthread_t *thread, const pthread_attr_t *attr,
■■    void *(*start_routine)(void*), void *arg) {
    int rc = pthread_create(thread, attr, start_routine, arg);
    assert(rc == 0);
}

void Pthread_join(pthread_t thread, void **value_ptr) {
    int rc = pthread_join(thread, value_ptr);
    assert(rc == 0);
}
#endif // __MYTHREADS_h__
```

File: pc-header.h

```
#ifndef __pc_header_h__
#define __pc_header_h__

#define MAX_THREADS (100)  // maximum number of producers/consumers

int producers = 1;         // number of producers
int consumers = 1;         // number of consumers

int *buffer;               // the buffer itself: malloc in main()
int max;                   // size of the producer/consumer buffer

int use_ptr  = 0;          // tracks where next consume should come from
int fill_ptr = 0;          // tracks where next produce should go to
int num_full = 0;          // counts how many entries are full

int loops;                 // number of items that each producer produces

#define EMPTY         (-2) // buffer slot has nothing in it
#define END_OF_STREAM (-1) // consumer who grabs this should exit

#endif // __pc_header_h__
```